

Program Designing 'and' { [Programming].ICT 1305

< J.M.D.S.Jayasinghe >

< Department of ICT >

< Faculty of Technology >

< Rajarata University of Sri Lanka >

}

{ C Programming }

What is C Programming Language {

- 'C' was developed by Dennis Ritchie in 1972.
- It is widely used in the software development field.
- It is a procedure and structure-oriented language.
- It has the full support of various operating systems and hardware platforms.
- Many compilers are available for executing programs written in 'C'.
- A compiler compiles the source file and generates an object file.
- A linker links all the object files together and creates one executable file.
- It is highly portable.

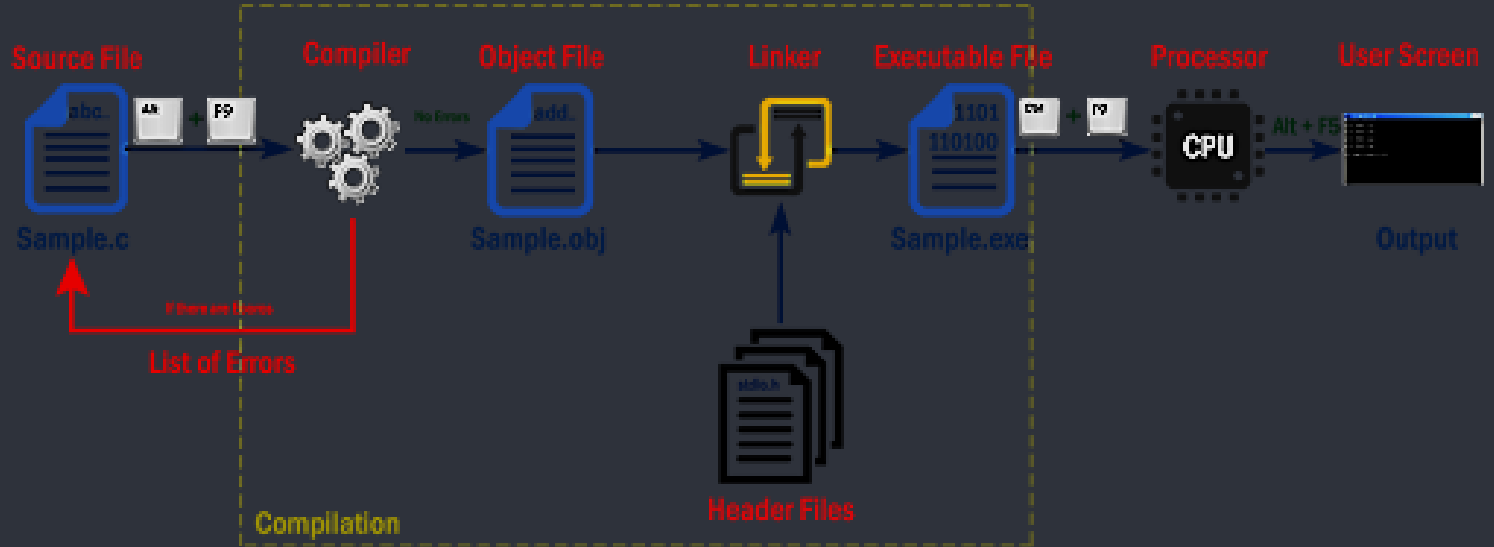
Key Applications {

- For creating desktop applications
- For developing games
- For designing operating systems
- For designing compilers
- For embedded systems
- For Network drivers

}

Working of C step by step procedure {

1. **Writing Source Code** – The first step is to write a source code for the desired project. The code is written using a text editing tool.
2. **Compiling the source code** – The source code is then converted into equivalent machine instructions, so that the computer can understand it.
3. **Checking for Errors** – The code is checked for all possible errors
4. **Linking** – The code is linked with other files of code. Sometimes it is difficult to put all your code in one file. So, the computer needs to link them together if you decided to separate them.
5. **Checking for errors** – The code is checked for all possible errors once again.
6. **Execution** – The code is then finally executed to get the desired output.



What is an IDE {

- An **Integrated Development Environment** (IDE) is software for building applications that combines common developer tools into a single graphical user interface (GUI). An IDE typically consists of:

1. Code Editor:

A text editor designed for programming. Features: syntax highlighting, auto-indentation, and code suggestions.

}

What is an IDE {

2. Compiler/Interpreter Integration:

Let's you compile or run code without leaving the IDE.
For C, it integrates with compilers like GCC or Clang.

3. Debugger:

Helps find and fix errors in your code.

You can step through code line by line, watch variables, and see program flow.

4. Build Automation: Automates compiling, linking, and running programs.

}

C Tokens {

- A token is an atomic unit (smallest indivisible units) in a program.
- The most basic elements in a C program recognized by the compiler are a single character or a group of characters called C tokens.
- The compiler cannot breakdown the token any further.
- For Example, the words main, '{' (brace), '(' (parenthesis) are all tokens of C program.

}

Types of Tokens {



}

C Token – Keywords {

- Keywords in C are reserved words that have predefined meanings and are part of the C language syntax.
- Keywords have special meaning to the C compiler. These keywords can be used only for their intended action; they cannot be used for any other purpose.
- These keywords cannot be used as variable names, function names, or any other identifiers within the program except for their intended purpose.
- They are used to define the structure flow and behavior of a C program.

`int, return, if, else, while, for, char, float`

}

C Token – Identifiers {

- Identifiers are used as the general terminology for the naming of variables, functions, and arrays.
- These are user-defined names consisting of sequence of letters and digits with either a letter or the underscore(_) as a first character.
- Identifier names must differ in spelling and case from any keywords. You cannot use keywords as identifiers.

}

C Token – Identifiers {

- Once declared, you can use the identifier in later program statements to refer to the associated value.
- Example: `age`, `sum_of_numbers`, `_temp`.
- Examples of Invalid Identifiers: `1variable` (starts with a digit), `total#value` (contains special character #), `float` (reserved keyword), `my var` (contains a space)

}

Rules for Naming Identifiers {

- The first character of an identifier should be either an alphabet or an underscore, and then it can be followed by any of the character, digit, or underscore.
- It should not begin with any numerical digit.
- Identifiers are case sensitive.
- Commas or blank spaces cannot be specified within an identifier.
- Keywords cannot be represented as an identifier.
- The length of the identifiers should not be more than 31 characters.
- Identifiers should be written in such a way that it is meaningful, short, and easy to read.

}

C Token – Constants {

- A constant is a value assigned to the variable which will remain the same throughout the program.
- The constant value cannot be changed.
- There are two ways of declaring constant:
 - Using `const` keyword
 - Using `#define` pre-processor

}

C Token – Constants {

Constant

Example

Integer constant

10, 11, 34, etc.

Floating-point constant

45.6, 67.8, 11.2, etc.

Octal constant

011, 088, 022, etc.

Hexadecimal constant

0x1a, 0x4b, 0x6b, etc.

Character constant

'a', 'b', 'c', etc.

String constant

"java", "c++", ".net", etc.

}

C Token – Operators {

- Operators are symbols that trigger an action when applied to C variables and other objects.
- The data items on which operators act are called operands.
- Depending on the number of operands that an operator can act upon, operators can be classified as follows:
 - **Unary Operators:** Those operators that require only a single operand to act upon are known as unary operators. Example increment (++) and decrement (--) operators

}

C Token – Operators {

- **Ternary Operator:** The operator that requires three operands to act upon is called the ternary operator. Conditional Operator(?) is also called the ternary operator.
- **Binary Operators:** Those operators that require two operands to act upon are called binary operators. Binary operators can further be classified into:
 - Arithmetic operators
 - Relational Operators
 - Logical Operators
 - Assignment Operators

}

C Token – Strings {

- Strings are nothing, but an array of characters ended with a null character (`'\0'`).
- This null character indicates the end of the string.
- Strings are always enclosed in double quotes.
- Whereas a character is enclosed in single quotes in C.
- Examples of String:
 - `char a[10] = "Rajarata";`
 - `char a[] = "Rajarata";`
 - `char a[10] = {'R','a','j','a','r','a','t','a','\0'};`

}

C Token – Special Characters {

- Special characters are symbols that have predefined meanings and are used to perform specific tasks in a program.
- **Brackets []**: Opening and closing brackets are used as array element references. These indicate single and multidimensional subscripts.
- **Parentheses ()**: These special symbols are used to indicate function calls and function parameters.

}

C Token – Special Characters {

- **Braces {}:** These opening and ending curly braces mark the start and end of a block of code containing more than one executable statement.
- **Comma (,):** It is used to separate more than one statement like for separating parameters in function calls.
- **Colon(:):** It is an operator that essentially invokes something called an initialization list.

}

C Token – Special Characters {

- **Semicolon(;) :** It is known as a statement terminator. It indicates the end of one logical entity. That's why each individual statement must be ended with a semicolon.
- **Asterisk (*) :** It is used to create a pointer variable and for the multiplication of variables.
- **Assignment operator(=) :** It is used to assign values and for logical operation validation.

}

C Token – Special Characters {

- **Pre-processor (#):** The preprocessor is a macro processor that is used automatically by the compiler to transform your program before actual compilation.
- **Period (.):** Used to access members of a structure or union.
- **Tilde(~):** Bitwise One's Complement Operator.

}

Structure of a C Program {

```
1  #include<stdio.h>
2
3  int main(){
4
5      /* my first program in C */
6
7      printf("Hello World \n");
8
9      return 0;
10 }
```

Header (Preprocessor)

Main Function

Comment

Print Function

Return Keyword

}

Structure of a C Program {

- `Comments` in the c program are optional and may appear anywhere in a C program.
- Comments are enclosed between `/*` and `*/` or `//`.
- `# include <stdio.h>` tells the compiler to read the file `stdio.h` and `include` its contents in this file.
- `stdio.h`, one of header files, contain the information about input and output functions.
- `int main( )` is the start of the main program.
- The left brace `{` represents the beginning of the program.
- The `"return 0"` statement also indicates that the program has been successfully compiled and run.
- the right brace `}` represents the ending of the program.

Rules to write a C program {

- All C statements must end with semicolon. ;
- C is case-sensitive. That is, upper case and lower-case characters are different. Generally, the statements are typed in lower case.
- A C statement can be written in one line, or it can split into multiple lines.
- Braces must always match upon pairs, Ex. every opening brace { must have a matching closing brace }.
- Every C program starts with main() function.
- Comments cannot be nested. For example, `/* Welcome to 'C' ,/* programming*/ */`
- A comment can be split into more than one line.

}

Variables {

- In programming, a variable is a container (storage area) to hold data.
- To indicate the storage area, each variable should be given a unique name (identifier).
- Variable names are just the symbolic representation of a memory location.
- We can store different types of data in the variable and reuse the same variable for storing some other data any number of times.
- C Variable Syntax : `data_type variable_name = value;`

Declaring & Initialization variables {

- Variable declaration in C tells the compiler about the existence of the variable with the given name and data type.
- Compiler allocates some memory and some value to it. A defined variable will contain some random garbage value till it is not initialized.

Ex: `float salary;`

- Initialization of a variable is the process where the user assigns some meaningful value to the variable when creating the variable.

Ex: `float salary = 300000;`

}

Rules for defining variables {

- Variable names include letters (uppercase and lowercase), digits, and underscores. They must start with a letter (uppercase or lowercase) or an underscore.
- C is a case-sensitive programming language. It means that uppercase and lowercase letters are considered distinct. For example, `myVar`, `MyVar` and `myvar` are all considered different variable names.
- Variable names cannot be the same as C keywords (reserved words), as they have special meanings in the language.
- Variable names cannot contain spaces or special characters (such as `!`, `@`, `#`, `$`, `%`, `^`, `&`, `*`, `(`, `)`, `-`, `+`, `=`, `[`, `]`, `{`, `}`, `|`, `\`, `/`, `<`, `>`, `.,`, `?,`, `;`, `'`, or `"`).

}

Data Types {

- Data types are declarations for variables. This determines the type and size of data associated with variables.
- A data type specifies the type of data that a variable can store such as integer, floating, character, etc.
- Each data type requires different amounts of memory and has some specific operations which can be performed over it.
- There are the following basic data types in C language.
- Ex: int, char, float, double, short int, long int, long double, void, etc.

}

Data Types {

- int - Integers are whole numbers that can have both zero, positive and negative values but no decimal values.
- char - Individual characters are represented by the char data type. Characters such as 'A', 'b', '5', or '\$' are enclosed in single quotes.
- float - Float data type is used to store floating-point (decimal and exponential) values with single precision.
- double - A Double data type store decimal numbers (numbers with floating point values) with double precision.

}

Format Specifiers {

- Format specifiers in C are used to take inputs and print the output of a type.
- The symbol we use in every format specifier is %.
- Format specifiers tell the compiler about the type of data that must be given or input and the type of data that must be printed on the screen.

}

Format Specifiers {

Specifier	Type
%d	int
%c	char
%s	char[]
%f	float
%lf	double

}

Escape Sequences {

- Escape sequences in C are special character combinations that allow you to represent nonprintable characters in strings and character literals, using a backslash (\) followed by specific characters.

}

Escape Sequences {

Escape Sequences	Meaning
\'	Single quote
\"	Double quote
\n	New line
\t	Tab
\b	Backspace
\\	Backslash

}

C Input Output (I/O) {

The printf() and scanf() functions are used for input and output in C language. Both functions are inbuilt library functions, defined in stdio.h (header file).

printf() function

- The printf() function is used for output. It prints the given statement to the console.

}

C Input Output (I/O) {

- The syntax of printf() function is, `printf("format string",argument_list);`
- The *format string* contains the following:
 - String that are simply printed on the screen.
 - Format specifiers that begin with a % sign and define the output format for display of each item.
 - Escape sequence characters.

}

C Input Output (I/O) {

scanf() function

- The scanf() function is used for input. It reads the input data from the console.
- The syntax of scanf() function is, `scanf("format string",argument_list);`
- Examples
 - `scanf("Emp number: %d", &empNumber );`
 - `printf("Salary: %f ", salary);`

}

Simple C Program {

Write a program to add two numbers given by the user.

```
1  #include<stdio.h>
2
3  int main(){
4
5      int Number_1, Number_2, Total;
6      printf("Enter Number 1: ");
7      scanf("%d", &Number_1);
8      printf("Enter Number 2: ");
9      scanf("%d", &Number_2);
10     Total = Number_1 + Number_2;
11
12     printf("Total: %d \n", Total);
13
14     return 0;
15 }
```

{ Thank You }